

Exploratory Combinatorial Optimization Problem Solving via Gauge Transformation

Tianle Pu^{1,*}, Changjun Fan^{1,*,†}, Mutian Shen^{2,*}, Yizhou Lu^{3,4,*}, Li Zeng^{1,*}
Zohar Nussinov^{2,5,6}, Chao Chen¹, Zhong Liu¹

¹ *Laboratory for Big Data and Decision, National University of Defense Technology, Changsha, China*

² *Department of Physics, Washington University, St. Louis, MO 63130, USA*

³ *Big Data and Risk Management Research Center, Beijing, China*

⁴ *University of International Business and Economics, Beijing, China*

⁵ *Rudolf Peierls Centre for Theoretical Physics, University of Oxford, UK*

⁶ *LPTMC, CNRS-UMR 7600, Sorbonne Université, 4 Place Jussieu, 75252 Paris cedex 05, France*

{putl22, fanchangjun, zengli24, chenc1997, liuzhong}@nudt.edu.cn, {smutian, zohar}@wustl.edu, litix.yz.lu@gmail.com

Abstract—The combinatorial optimization problems (COPs) over graph are of great significance both in theory and practice, covering a wide range of scenarios in daily life and industrial production. Recent years, reinforcement learning (RL) based models have emerged as a promising direction, which treat solving the COPs as a heuristic learning problem. However, current finite-horizon Markov Decision Process (MDP) based RL models are not allowed to explore adequately for improving solutions at test time, which may be necessary given the complexity of NP-hard optimization tasks. Some recent attempts solve this issue by focusing on reward design and state feature engineering, which are tedious and ad-hoc. To address this challenge, we introduce a physics-inspired technique called gauge transformation (GT), which is highly effective in enabling RL agents to explore and continuously enhance solution quality during testing. GT seamlessly transforms any state within the MDP back to its initial state, allowing the RL agent to continue exploration within the transformed space. Empirically, we demonstrate that traditional RL models equipped with the GT technique achieve the SOTA performance on the MaxCut problem. Moreover, GT is exclusively applied during testing and does not alter the training phase of the model. It can be readily integrated into existing RL models, providing a pathway for more effective exploration in solving the COPs.

Index Terms—Combinatorial Optimization, Gauge Transformation, Graph Neural Network, Reinforcement Learning, MaxCut Problem

I. INTRODUCTION

Combinatorial optimization problems (COPs) over graph typically involve seeking the optimal combination of nodes and edges in the graph to reach a certain optimal goal. Due to the expansive nature of the solution space, these problems are frequently classified as NP-hard [1]. Over the course of several decades, extensive research in this field has given rise to three primary methods: exact algorithms [2], approximation algorithms [3], [4], and heuristics [5]. However, these methods often encounter difficulties in balancing computational efficiency with solution precision, especially when dealing with large-scale problems.

The rise of deep learning has led to neural-based solvers for COPs. These solvers treat COPs as learning tasks [6], [7]. Deep reinforcement learning (DRL), especially, has advanced

by using graph embeddings to develop efficient heuristics. DRL methods are divided into two groups: learning improvement heuristics (LIH) [8] and learning constructive heuristics (LCH) [6], [7], [9]. LIH generates the full solution set at once, while LCH builds the solution by sequentially adding nodes. However, LCH methods do not revisit selected nodes, resulting in a single “best guess” that may not be optimal given the complexity of COPs.

An intuitive way to further improve the model’s performance would be to enable the agent to keep revisiting the nodes and to add or remove them from the solution subset until no further improvement can be made. This ever-improving adjustment can be easily done by heuristics like the simulated annealing [10], which rely on sampling and iterative refinement. Yet, the same adjustment is challenging for the finite-horizon MDP (Markov Decision Process) based reinforcement learning (RL) models. Since such RL algorithms always start from the same configuration, and end at the same configuration, the agent always finishes the MDP within finite steps. However, the inherent large solution space of COPs makes it challenging to reach the global optima via only one single exploration. Therefore, one model that can explore the solution space at test time and seek ever-improving states is needed. Recently, researchers have also noticed this issue and made some inspiring attempts. For instance, they devote to designing more complicated network architectures to expand the horizon of finite MDP [7], [9]. However, these improvements are often ad-hoc, making them difficult to apply in other COPs.

In this work, we instead employ a technique for the RL agent during inference without any changes on the model architecture. The technique, Gauge Transformation (GT) [11], could effectively allow the trained agent reverse its earlier actions during inference stage. GT achieves this by finding equivalent representations for the same problem and then mapping the current state to the same configuration as the initial one, in this way, the agent is able to continually seek improving solutions. In conjunction with the LCH-based methods, GT serves as a straightforward plugin that effectively enhances the performance of the original model without any modifications.

We conduct experiments on the MaxCut problem [12], a canonical NP-complete COP to which many COPs can be reduced. Extensive calculations on both synthetic and real-world datasets show that GT+S2V-DQN (named S2V-DQN-GT) brings great effort over traditional RL models.

Our contributions can be summarized as follows:

- To the best of our knowledge, GT is the first to employ the “transform then explore” scheme to overcome the bottleneck that existing RL agents can only conduct one-single exploration, and it is supported by robust theoretical foundations.
- We are mathematically analyzing the theoretical reasons for the success of GT and providing the applicable scope of GT.
- S2V-DQN-GT demonstrates significantly better performance and generalization ability on the MaxCut problem.

II. PRELIMINARIES

A. Combinatorial Optimization

Combinatorial optimization challenges involve making selections within scenarios where numerous binary decisions are necessary, and each combination of choices results in a specific objective function value [13], [14]. In practice, we could encode non-binary variables, such as integer variables, from the original COP using binary variables [14], [15]. Formally, a binary COP is represented as follows:

$$\begin{aligned} x^* &= \arg \min_{x \in \mathcal{D}} f(x) \\ \text{s.t. } h_m(x) &\leq 0, m = 1, \dots, M \\ g_l(x) &= 0, l = 1, \dots, L \end{aligned} \quad (1)$$

where x devotes binary variables, $f(x)$ is the objective function, $\mathcal{D} = \{-1, +1\}^n$ is a binary decision space satisfying the given inequality and equality constraints $h_m(x)$ and $g_l(x)$. Furthermore, by employing penalty function method, Eq. 1 is transformed into unconstrained one and could be rewritten as follows:

$$x^* = \arg \min_{x \in \mathcal{D}} f(x) + \lambda_1 \max[h(x), 0] + \lambda_2 \|g(x)\|^2 \quad (2)$$

where $\lambda_1, \lambda_2 > 0$ serve as penalty terms for inequality and equality constraints and $\|\cdot\|$ denotes the norm. For clarity, we denote the objective function of Eq. 2 as $H(x)$ and express it in vector form:

$$\begin{aligned} x^* &= \arg \min_{x \in \mathcal{D}} H(x) = \mathbf{c}^T \mathbf{x} \\ &= \sum_{k=0}^N \sum_{\langle i_1, i_2, \dots, i_k \rangle} c_{i_1 i_2 \dots i_k} x_{i_1} x_{i_2} \dots x_{i_k} \end{aligned} \quad (3)$$

where $\langle i_1, i_2, \dots, i_k \rangle$ represents a set of k binary variables, and the cost associated with the interactions of these variables is denoted as $c_{i_1 i_2 \dots i_k}$. Naturally, we can use $\{c_{i_1 i_2 \dots i_k}, x_{i_k}\}$ to represent a COP instance. Therefore, a binary COP can be represented on an undirected graph $G = (V, E, W)$, where V is the set of nodes composed of variables, E is the set of edges representing whether there are interactions between variables, and W is the set of edge weights representing the costs of interactions between variables.

B. LCH-based neural solver

Typically, a feasible solution to a COP, denoted by $\mathbf{x} = (x_1, x_2, \dots)$, where x_i represents a yes/no decision (e.g., whether a node is included in the cut-set in the MaxCut problem). The LCH-based neural solver iteratively adds nodes to the solution set and outputs the corresponding set as the optimal solution, thereby maximizing or minimizing the objective function during construction. And the complete process is based on the same MDP $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}\}$ [6], [7], [9], represented as follows:

State. The state at step i , denoted as s_i , represents that i nodes have been added to the solution set. Typically, nodes in the initial configuration are set to $+1$, whereas those in the terminal configuration are set to -1 .

Action. The action is to select a node. In other words, it flips the state of the chosen node to -1 .

Reward. The reward is the change in the objective function resulting from adding the i -th selected node to the solution set at step i .

Policy. Specially, a deep neural network Q_θ can be trained to guide the node selection [6], [7]. Consequently, the policy π_θ can be determined by: $\pi_\theta(s_i) = \arg \max_{a_i} Q_\theta(s_i, a_i)$.

III. GAUGE TRANSFORMATION TECHNIQUE

A. GT for COPs

1) *The concept of GT:* The GT is a transformation operation that keeps the objective function value unchanged, and all GT transformations collectively form a group known as the GT group, defined as follows:

Definition 1 (GT Group). Suppose that $H : \{-1, +1\}^n \rightarrow \mathbb{R}$ is a function. The set of all transformations g that preserve the objective function value $H(x)$ forms the GT group.

$$G_H = \{g \mid H(g(x)) = H(x)\}$$

Specifically, the transformation $g \in G_H$ modifies a given COP instance $\{c_{i_1 i_2 \dots i_k}, x_{i_k}\}$ as follows:

$$\begin{aligned} g(c_{i_1 i_2 \dots i_k}) &= c_{i_1 i_2 \dots i_k} t_{i_1} t_{i_2} \dots t_{i_k} \\ g(x_{i_k}) &= x_{i_k} t_{i_k} \end{aligned} \quad (4)$$

where t_{i_k} are independent auxiliary variables such that x_{i_k} could take a desired value. This technique switches the COP instance between any two configurations while maintaining the value of the objective function. Recall that the initial state of the LCH neural solver typically sets the variable configuration to $+1$. Of course, the GT also allows any random input configuration to be transformed into the $+1$ configuration. This implies that if the variable $x_{i_k} = +1$, its auxiliary variable $t_{i_k} = +1$; conversely, if $x_{i_k} = -1$, its auxiliary variable $t_{i_k} = -1$. In this way, GT enables any LCH-based neural solver to process arbitrary random input configurations of variables.

Furthermore, we will illustrate the technique of GT on a graph. In fact, GT switches any node state to $+1$ and transfers the impact of the switch on the objective function value to the weights of the edges, as shown in 1.

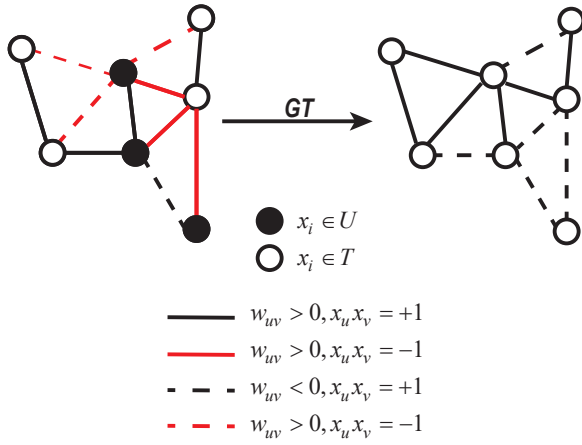


Fig. 1. The illustration of GT on a graph.

2) *The scope of GT's applicability:* Before using GT, it is imperative to clarify the scope of its applicability. Generally, GT is suitable for the following two major categories of cases:

- Mathematical formulations are akin to Eq. 3 in COPs. These problems are typically unconstrained and driven by edge weights, such as MaxCut [12], k-SAT [16], and others.
- Transforming the problems into a form similar to Eq. 3. This category generally covers most COPs over graphs. However, problem transformation is not the focus of this paper. Please refer to [14], [15] for more details.

Therefore, GT is a simple and efficient technique that can be applied to most COPs over graphs.

B. Potential reasons behind GT's success

In this section, we explain the potential reasons why GT helps improve the performance of LCH-based neural solvers for COPs. Generally, the success of GT hinges on the following two important properties.

Theorem III.1 (Transformation invariance). *Suppose that $H: \{+1, -1\}^n \rightarrow \mathbb{R}$ is a function, $g \in G_H$ is a GT transformation. Then, $g \circ H = H$*

Proof By Definition 1 and Eq. 4, we have

$$\begin{aligned}
 g \circ H &= \sum_{k=0}^N \sum_{\langle i_1, i_2, \dots, i_k \rangle} c_{i_1 i_2 \dots i_k} t_{i_1} t_{i_2} \dots t_{i_k} x_{i_1} t_{i_1} x_{i_2} t_{i_2} \dots x_{i_k} t_{i_k} \\
 &= \sum_{k=0}^N \sum_{\langle i_1, i_2, \dots, i_k \rangle} c_{i_1 i_2 \dots i_k} x_{i_1} x_{i_2} \dots x_{i_k} t_{i_1}^2 t_{i_2}^2 \dots t_{i_k}^2
 \end{aligned}$$

Substituting $t_{i_k}^2 = 1$ into $g \circ H$ yields

$$\begin{aligned}
 g \circ H &= \sum_{k=0}^N \sum_{\langle i_1, i_2, \dots, i_k \rangle} c_{i_1 i_2 \dots i_k} x_{i_1} x_{i_2} \dots x_{i_k} \\
 &= H
 \end{aligned}$$

So we complete the proof.

Theorem III.2 (Submodularity). *Suppose that $H: \{+1, -1\}^n \rightarrow \mathbb{R}$ is a function, $g \in G_H$ is a GT transformation, and π_θ^* is the deterministic policy trained by the LCH-based neural solver. Then, $\pi_\theta^*(g \circ H^{(t+1)}) \geq \pi_\theta^*(H^{(t)})$*

Proof From the Bellman optimality equation, we have $\pi_\theta^*(H^{(t+1)}) \geq \pi_\theta^*(H^{(t)})$. Combining $g \circ H = H$, we can derive $\pi_\theta^*(H^{(t+1)}) = \pi_\theta^*(g \circ H^{(t+1)})$. So we complete the proof.

Theorem III.1 states that GT maintains the value of the objective function unchanged, while Theorem III.2 shows that after the GT transformation, the optimal solution found by the LCH-based neural solver in subsequent explorations is always at least as good as the solution found in the previous round. This reveals the underlying reason for GT's success.

C. The strength of GT

In this section, we will illustrate the strength of GT by combining it with the MaxCut problem. Before that, recall the MaxCut problem together.

MaxCut. The MaxCut Problem aims to partition the vertices of a graph into two separate subsets, with the goal of maximizing the cumulative weight of the edges that are severed between the subsets. Formally, Given a graph, $G(V, E, W)$, the MaxCut problem is to find a subset of nodes $T \subseteq V$ that maximizes the sum of edge weights in the cut set $C \subseteq E$, where the edges (u, v) in C satisfy $u \in T$ and $v \in V \setminus T$. The optimising function can be written as:

$$H(u, v) = \sum_{(u, v) \in C} W_{uv} \quad (5)$$

State resetting and revisiting. As shown in Fig. 2, GT helps improve the performance of the LCH-based neural solver mainly through two steps: First, the neural solver explores one round to find the optimal solution. Then, using the GT transformation, the neural solver explores again in the transformed space. The best solution found in this exploration is then transformed back to the original problem. If the optimal solution found after the GT transformation is better than the solution found in the previous round, the above actions are repeated until the GT can no longer improve the quality of the solution.

multiple exploration from different initial configurations. GT can switch the configuration of any input variables to the initial configuration, allowing the neural solver to start exploring from multiple starting points. This is akin to a robot being able to choose multiple paths from the base of a mountain, significantly increasing the likelihood that the neural solver will find the optimal solution. The application of GT has resolved the dilemma faced by the LCH-based neural solver, which was limited by a narrow horizon and could only start exploration from one initial state.

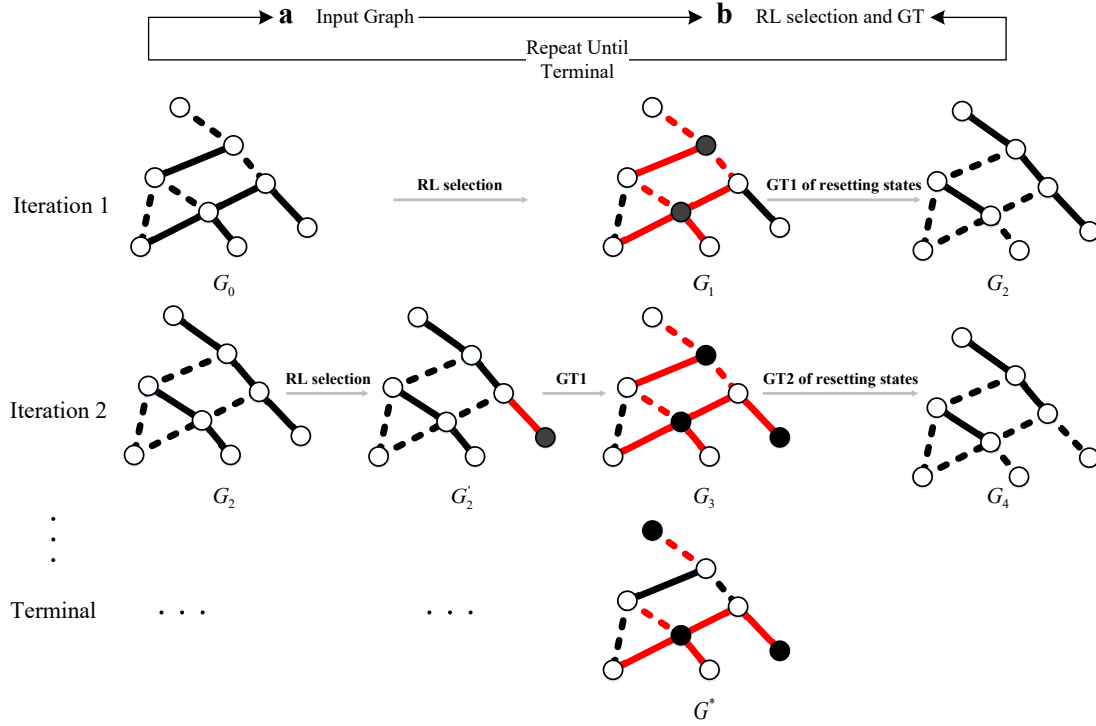


Fig. 2. The illustration of GT on a graph. White and black nodes denote $u \in U \equiv V \setminus T$ and $u \in T$ respectively. Solid and dashed lines correspond to the weights of the edges are positive and negative.

IV. EXPERIMENTS

In this section, we employ the Max-Cut problem to demonstrate the advantages of GT in addressing COPs. Our primary goal is to answer the following questions:

RQ1: How are GT-enhanced RL models compared with other competing methods?

RQ2: How to utilize GT best in practice?

A. Experimental Setup

1) *Datasets.*: we perform experiments using both synthetic and real-world datasets. For the synthetic data, we utilize three established types of graphs: ER graphs [17], BA graphs [18], and WS graphs [19]. These graphs represent different characteristics typically found in network structures. Moreover, we assign edge weights according to three different distributions: a uniform distribution $\mathcal{U}(0, 1)$ ranging from 0 to 1, a normal distribution $\mathcal{N}(0, 1)$, and a bimodal distribution $\mathcal{D}\{-1, 0, +1\}$. for the real-world data, We test on two real-world datasets, namely ‘Physics’ and Gset [20] datasets. The ‘Physics’ encompasses a total of 10 graphs, each comprising 125 nodes and 375 edges. And the Gset serves as a highly regarded benchmark in graph research. In our work, we choose the top 10 graphs from Gset, which exhibit a substantial scale with 800 nodes. Notably, the edge weights of all graphs in these real-world datasets conform to the bimodal $\mathcal{D}\{-1, 0, +1\}$ distribution.

2) *Baseline methods.*: We compare our method (S2V-DQN-GT) with a wide range of competing methods for the Max-Cut problem, including greedy method, i.e. *MaxcutApprox*(MCA) [21], and RL-based methods, including S2V-DQN [6] and ECO-DQN [7]. Note that our proposed new method, S2V-DQN-GT, simply repeats several iterations of S2V-DQN, with the guide of GT, until the objective value can no longer be optimized.

3) *Evaluation metrics.*: To maintain consistency with prior research, the approximate ratio $AR(s^*)$ refers to the average ratio of the results obtained by each baseline method on the test set compared to the results achieved by the Gurobi solver [22] within 3600 seconds.

4) *Other settings.*: All of our hyperparameters are set consistent with those of S2V-DQN, without any task-specific handcrafted tuning. We believe that with an exhaustive search, the model would demonstrate even better performance. All experiments are performed on the same system with an Nvidia GeForce Tesla V100-32GB.

B. Results

1) *RQ1: How does GT-enhanced RL compare with competing methods?*: It is necessary to understand how GT improves performance in relation to the current SOTA RL-based approach. To ensure a fair evaluation, each model is trained on the BA graphs containing 50-100 nodes with an average degree of 4. Notably, some methods, like S2V-DQN, are limited to starting from a specific state distribution. In order to maintain consistency, the initial node state for all methods was uniformly

TABLE I
COMPARISON RESULTS OF DIFFERENT BASELINE MODELS UNDER THREE DISTRIBUTIONS. THE BEST RESULTS ARE IN BOLD, WHILE THE SECOND-BEST ONES ARE UNDERLINED.

(a) $\mathcal{U}(0, 1)$					
	50-100	100-200	200-300	300-400	400-500
MCA	0.949 $\pm$ 0.004	0.943 $\pm$ 0.002	0.941 $\pm$ 0.002	0.939 $\pm$ 0.002	0.941 $\pm$ 0.002
S2V-DQN	0.951 $\pm$ 0.003	0.946 $\pm$ 0.002	0.944 $\pm$ 0.002	0.940 $\pm$ 0.002	0.943 $\pm$ 0.002
ECO-DQN	0.989 $\pm$ 0.002	0.984 $\pm$ 0.001	0.970 $\pm$ 0.001	0.958 $\pm$ 0.001	0.933 $\pm$ 0.003
MCA-GT	0.958 $\pm$ 0.003	0.954 $\pm$ 0.002	0.952 $\pm$ 0.002	0.950 $\pm$ 0.002	0.950 $\pm$ 0.002
S2V-DQN-GT	0.990 $\pm$ 0.002	0.980 $\pm$ 0.002	0.978 $\pm$ 0.002	0.970 $\pm$ 0.001	0.968 $\pm$ 0.001

(b) $\mathcal{N}(0, 1)$					
	50-100	100-200	200-300	300-400	400-500
MCA	0.868 $\pm$ 0.009	0.850 $\pm$ 0.006	0.849 $\pm$ 0.005	0.852 $\pm$ 0.004	0.844 $\pm$ 0.004
S2V-DQN	<u>0.936 $\pm$ 0.007</u>	<u>0.909 $\pm$ 0.005</u>	<u>0.879 $\pm$ 0.005</u>	0.858 $\pm$ 0.005	0.821 $\pm$ 0.007
ECO-DQN	0.928 $\pm$ 0.006	0.901 $\pm$ 0.005	0.877 $\pm$ 0.004	0.844 $\pm$ 0.005	0.800 $\pm$ 0.007
MCA-GT	0.880 $\pm$ 0.009	0.859 $\pm$ 0.006	0.857 $\pm$ 0.005	<u>0.861 $\pm$ 0.004</u>	<u>0.852 $\pm$ 0.004</u>
S2V-DQN-GT	0.987 $\pm$ 0.003	0.973 $\pm$ 0.004	0.955 $\pm$ 0.004	0.934 $\pm$ 0.004	0.925 $\pm$ 0.003

(c) $\mathcal{D}\{-1, 0, +1\}$					
	50-100	100-200	200-300	300-400	400-500
MCA	0.838 $\pm$ 0.010	0.814 $\pm$ 0.005	0.814 $\pm$ 0.006	0.803 $\pm$ 0.005	0.806 $\pm$ 0.004
S2V-DQN	0.925 $\pm$ 0.006	0.912 $\pm$ 0.006	<u>0.900 $\pm$ 0.004</u>	<u>0.873 $\pm$ 0.005</u>	<u>0.851 $\pm$ 0.004</u>
ECO-DQN	<u>0.934 $\pm$ 0.005</u>	<u>0.912 $\pm$ 0.006</u>	0.890 $\pm$ 0.004	0.858 $\pm$ 0.006	0.810 $\pm$ 0.008
MCA-GT	0.854 $\pm$ 0.008	0.824 $\pm$ 0.006	0.827 $\pm$ 0.007	0.812 $\pm$ 0.005	0.819 $\pm$ 0.005
S2V-DQN-GT	0.976 $\pm$ 0.004	0.965 $\pm$ 0.004	0.955 $\pm$ 0.003	0.942 $\pm$ 0.004	0.925 $\pm$ 0.003

set to ‘+1’. Besides, the number of initial states of ECO-DQN is 1. As a simple and efficient technique, GT can also be used in MCA, named MCA-GT. Table I summarizes the results of each method under different distributions. Besides, Table II shows the performance of different methods on real-world datasets.

TABLE II
COMPARISON RESULTS OF DIFFERENT BASELINE MODELS ON REAL-WORLD DATASET.

	Physics	Gset
S2V-DQN	0.894 $\pm$ 0.012	0.904 $\pm$ 0.020
ECO-DQN	0.988 $\pm$ 0.005	0.983 $\pm$ 0.003
S2V-DQN-GT	0.986 $\pm$ 0.006	0.978 $\pm$ 0.006

Compared with other methods, S2V-DQN-GT achieves a notably elevated level of accuracy across all distributions. Conversely, ECO-DQN encounters challenges in sustaining accuracy when confronted with varying distributions or extensive graph sizes, due to its employment of targeted exploratory techniques to augment state exploration. Regarding MCA, GT demonstrates a certain degree of accuracy improvement, albeit not significantly pronounced. This is attributed to the fact that the selection of actions in the MCA method relies on the present state, rendering the long-term rewards of actions uncertain. Such ambiguity represents a distinctive advantage of RL-based methods.

2) *RQ2: How to utilize GT best in practice?*: In order to further elucidate the generalization capacity of GT under different graph sizes, we train the model in BA graphs under $\mathcal{U}(0, 1)$. Table III presents compelling evidence indicating that incorporating GT substantially enhances the model’s ability to generalize. Remarkably, the model exhibits consistently supe-

rior accuracy even when confronted with scenarios featuring 500-600 nodes.

As previously mentioned, the key advantage of GT resides in its capability to initiate exploration from distinct initial states through transformation. MCA can also be initialized from different initial states, simply by constructing the solution from a random set instead of an empty set. Importantly, we have observed that ECO-DQN incorporates techniques like Monte Carlo tree search to generate the random initial states. In order to maintain fairness, we make it a point to generate the identical random initial states for all methods. The results of these experiments are presented in Table V.

Through the results, it is evident that the utilization of a random initial states significantly enhances the accuracy of various approaches, encompassing both the greedy method and the RL-based method. Notably, the RL-based method demonstrates a notably more substantial improvement, such as S2V-DQN-GT is near optimal on the small scale graph, owing to its inclination to prioritize actions that yield greater long-term returns throughout the decision-making process.

Finally, a comparative analysis was performed to evaluate the computational time of different algorithms utilized on the BA graphs(average degree 4) with edge wights distribution of $\mathcal{N}(0, 1)$. The specific results of this comparative analysis are provided in Table 6. Notably, it is important to mention that in the comparison process, no modifications were made to the other methods themselves when applying the GT approach. As a consequence, a marginal increment in the overall execution time was observed for the broader context. However, this increment still remained in close proximity to the running time of the conventional greedy method. Remarkably, this marginal increment in time was counterbalanced by a significant enhancement in accuracy.

TABLE III
S2V-DQN-GT'S GENERALIZATION IN BA GRAPHS(AVERAGE DEGREE 4).

test train	15-20	40-50	50-100	100-200	200-300	300-400	400-500	500-600
15-20	0.991 ± 0.003	0.984 ± 0.002	0.979 ± 0.002	0.969 ± 0.002	0.967 ± 0.002	0.960 ± 0.002	0.960 ± 0.001	0.961 ± 0.002
40-50		0.989 ± 0.002	0.983 ± 0.002	0.972 ± 0.002	0.967 ± 0.002	0.958 ± 0.002	0.958 ± 0.001	0.951 ± 0.002
50-100			0.989 ± 0.002	0.980 ± 0.002	0.978 ± 0.002	0.970 ± 0.001	0.968 ± 0.001	0.966 ± 0.001
100-200				0.979 ± 0.002	0.967 ± 0.002	0.963 ± 0.001	0.956 ± 0.002	0.956 ± 0.001
200-300					0.981 ± 0.001	0.979 ± 0.001	0.977 ± 0.002	0.978 ± 0.001

TABLE IV
RUNNING TIME OF DIFFERENT METHODS. THE TIME MEASUREMENTS PRESENTED ARE EXPRESSED IN SECONDS (S).

	50-100	100-200	200-300	300-400	400-500
MCA	0.477 ± 0.0422	5.204 ± 0.447	25.649 ± 1.249	56.987 ± 2.212	119.193 ± 3.443
S2V-DQN	0.888 ± 0.0446	3.326 ± 0.189	10.249 ± 0.392	<u>19.018 ± 0.562</u>	<u>34.511 ± 0.981</u>
ECO-DQN	0.561 ± 0.0383	6.126 ± 0.406	11.344 ± 0.321	11.396 ± 0.316	11.317 ± 0.306
MCA-GT	0.628 ± 0.056	6.465 ± 0.502	27.665 ± 1.352	57.860 ± 2.169	122.311 ± 3.434
S2V-DQN-GT	3.881 ± 0.248	16.750 ± 1.083	49.771 ± 2.742	98.052 ± 5.081	182.352 ± 9.445

TABLE V
GT PERFORMANCE WITH DIFFERENT INITIAL STATES IN BA GRAPHS
UNDER $\mathcal{N}(0,1)$.

	50-100	100-200
MCA-GT (m=10)	0.918 ± 0.006	0.900 ± 0.005
MCA-GT (m=100)	0.925 ± 0.006	0.901 ± 0.005
ECO-DQN (m=10)	0.948 ± 0.005	0.921 ± 0.004
ECO-DQN (m=100)	0.988 ± 0.002	0.958 ± 0.003
S2V-DQN-GT (m=10)	<u>0.994 ± 0.001</u>	<u>0.980 ± 0.003</u>
S2V-DQN-GT (m=100)	0.995 ± 0.001	0.982 ± 0.002

V. CONCLUSION

In conclusion, our proposed method GT has successfully addressed the challenges faced by RL in tackling COPs in recent years. GT has proven to be a simple but effective technique that addresses the agent's exploration of the state, significantly enhancing the model's accuracy and demonstrating stable performance in generalization. Furthermore, we have provided a thorough theoretical analysis of GT, outlining its applicable scope and the mathematical principles behind its transformation. GT can be seamlessly integrated as a plug-in into a broader RL-based framework, opening up new avenues for solving COPs.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (NSFC, 72421002, 72025405, 62206303), Science and Technology Innovation Program of Hunan Province (2023RC3009).

REFERENCES

- [1] R. KARP, "Reducibility among combinatorial problems," *Complexity of Computer Computations*, pp. 85–103, 1972.
- [2] E. L. Lawler and D. E. Wood, "Branch-and-bound methods: A survey," *Operations research*, vol. 14, no. 4, pp. 699–719, 1966.
- [3] D. P. Williamson and D. B. Shmoys, *The design of approximation algorithms*. Cambridge university press, 2011.
- [4] V. V. Vazirani, *Approximation algorithms*. Springer, 2001, vol. 1.
- [5] P. Festa, "A brief introduction to exact, approximation, and heuristic algorithms for solving hard combinatorial optimization problems," in *2014 16th International Conference on Transparent Optical Networks (ICTON)*. IEEE, 2014, pp. 1–20.

- [6] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," *Advances in neural information processing systems*, vol. 30, 2017.
- [7] T. Barrett, W. Clements, J. Foerster, and A. Lvovsky, "Exploratory combinatorial optimization with reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3243–3250.
- [8] Y. Wu, W. Song, Z. Cao, J. Zhang, and A. Lim, "Learning improvement heuristics for solving routing problems," *IEEE transactions on neural networks and learning systems*, vol. 33, no. 9, pp. 5057–5069, 2021.
- [9] R. Sun, Z. Zheng, and Z. Wang, "Learning encodings for constructive neural combinatorial optimization needs to regret," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 18, 2024, pp. 20 803–20 811.
- [10] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, "Optimization by simulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [11] Y. Ozeki, "Gauge transformation for dynamical systems of ising spin glasses," *Journal of Physics A: Mathematical and General*, vol. 28, no. 13, p. 3645, 1995.
- [12] M. X. Goemans and D. P. Williamson, "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming," *Journal of the ACM*, vol. 42, no. 6, p. 1115–1145, Nov 1995. [Online]. Available: <http://dx.doi.org/10.1145/227683.227684>
- [13] M. J. Schuetz, J. K. Brubaker, and H. G. Katzgraber, "Combinatorial optimization with physics-inspired graph neural networks," *Nature Machine Intelligence*, vol. 4, no. 4, pp. 367–377, 2022.
- [14] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics i: a tutorial on formulating and using qubo models," *4or*, vol. 17, no. 4, pp. 335–371, 2019.
- [15] A. Lucas, "Ising formulations of many np problems," *Frontiers in physics*, vol. 2, p. 74887, 2014.
- [16] B. Molnár, F. Molnár, M. Varga, Z. Toroczkai, and M. Ercsey-Ravasz, "A continuous-time maxsat solver with high analog performance," *Nature communications*, vol. 9, no. 1, p. 4864, 2018.
- [17] P. Erdős, A. Rényi *et al.*, "On the evolution of random graphs," *Publ. math. inst. hung. acad. sci.*, vol. 5, no. 1, pp. 17–60, 1960.
- [18] R. Albert and A.-L. Barabási, "Statistical mechanics of complex networks," *Reviews of modern physics*, vol. 74, no. 1, p. 47, 2002.
- [19] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *nature*, vol. 393, no. 6684, pp. 440–442, 1998.
- [20] U. Benlic and J.-K. Hao, "Breakout local search for the max-cut problem," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 3, pp. 1162–1173, 2013.
- [21] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.
- [22] L. Gurobi Optimization, "Gurobi optimizer reference manual (gurobi optimization, llc)," 2022.